

# EGR6301 — Етичний хакінг

## Лабораторна робота 7: Повний ланцюжок атаки та захисний аналіз

### *Symfonos 1 — Від SMB-зачіпки до root та назад до захисника*

Факультет комп'ютерних технологій та кібербезпеки • Весна 2026

|                 |                                                                                                           |
|-----------------|-----------------------------------------------------------------------------------------------------------|
| Тривалість:     | 100 хвилин                                                                                                |
| Всього:         | 100 балів (Трек А: 75 + Трек В: 25)                                                                       |
| Цільова машина: | Symfonos 1 (VulnHub)                                                                                      |
| Download:       | <a href="https://www.vulnhub.com/entry/symfonos-1,322/">https://www.vulnhub.com/entry/symfonos-1,322/</a> |
| Тип роботи:     | Дворівнева: повний ланцюжок атаки + захисний аналіз                                                       |

#### ЯК ПРАЦЮЄ ЦЯ ЛАБОРАТОРНА РОБОТА

Ця лабораторна робота поділена на два треки. Трек А (Частини 1–4) — базова лінія на 75 балів. Він обов'язковий для всіх студентів: це повний ланцюжок атаки від розвідки до root-доступу. Якщо ви якісно виконаєте Трек А, ви отримаєте оцінку в діапазоні B+ — A-.

Трек В (Частини 5–7) — розширення на 25 балів для тих, хто хоче оцінку А. Він вимагає здатності подивитися на власну атаку очима захисника: що саме побачив би захисник у логах, які конфігураційні зміни зламали б вашу атаку, і де залишається сліпа зона. Трек В не можна виконати, скопіювавши walkthrough, Він вимагає ваших власних даних з Треку А.

Трек В оцінюється лише за умови якісного виконання Треку А. Якщо Трек А виконано нижче 50/75, роботу Треку В буде проігноровано, тому що немає даних, на які можна було б спиратися.

#### ОБОВ'ЯЗКОВІ ПРАВИЛА ЗВІТУВАННЯ

Якщо ваш звіт містить лише вивід інструментів і скріншоти без вашого власного аналізу, ви отримаєте НУЛЬ балів за цей розділ. Кожен скріншот має супроводжуватися абзацом: «ЩО ви знайшли», «ЧОМУ це важливо», «ЩО далі».

Ви ПОВИННІ задокументувати щонайменше дві реальні невдачі: інструмент, який помилився; сервіс, який виглядав перспективним, але нічого не дав; хибна гіпотеза, яка змусила вас витратити 15 хвилин даремно. Опишіть, що сталося, чому і як ви змінили підхід. Symfonos 1

навмисно побудована з оманливими сервісами. Якщо у вашому звіті немає жодної невдачі, це означає, що ви або не пробували достатньо широко, або скопіювали walkthrough.

Кожен студент повинен написати унікальне пояснення ключового переходу в ланцюжку атаки від моменту, коли одна знахідка відкрила наступний крок. Двоє студентів можуть отримати root на одній і тій самій цілі, але жодні двоє не повинні описувати свій процес міркування ідентично.

Трек В особливо чутливий до копіювання. Sigma-правило, засноване на загальних патернах LFI з інтернету, отримує максимум 3 з 10 балів. Правило повинно виявляти ТОЧНО ту послідовність подій, яку ви самі згенерували у логах цієї конкретної машини.

### Сценарій

Невелика грецька туристична агенція Symfonos Travel звернулася до вашої компанії з проханням виконати повний тест на проникнення одного з їхніх внутрішніх серверів. За попереднім описом клієнта, на сервері розгорнуто простий веб-сайт агенції, внутрішній файловий обмін для співробітників і кілька службових сервісів. Сервер не оновлювали давно, але внутрішня ІТ-команда вважає його "простим" і не очікує серйозних проблем.

Ваш контракт покриває повний ланцюжок: розвідка, експлуатація, ескалація привілеїв. На відміну від попередньої оцінки (Lab 6), цього разу ви ДОЗВОЛЕНО отримати root. Але цей контракт має особливість: клієнт платить не лише за отримання доступу. Він просить у звіті описати, що саме він побачив би у своїх логах під час вашої атаки, і рекомендувати 3 конкретні конфігураційні зміни, які б зупинили вас. Це розширення score робить цей контракт приблизно вдвічі дорожчим за звичайний пентест, і саме тому ваша компанія погодилася. Це трек В вашого звіту.

Technical briefing: цільовий сервер Symfonos 1, Debian-based Linux. Symfonos навмисно побудована як багатoshарова ціль з оманливими сервісами. Не всі відкриті порти ведуть до реальних знахідок. Вашим першим бажанням може бути "запустити Hydra проти SSH" або "спробувати LFI у кожному параметрі" — і в обох випадках ви витратите час даремно. Ваше завдання є читати інформацію, яку вам дає ціль, а не слідувати загальним патернам.

### Структура роботи

Усі частини 1–4 (Трек А) є обов'язковими для кожного студента. Трек В (Частини 5–7) додатковий для тих, хто претендує на оцінку А або А-. Орієнтовний розподіл часу — 70 хвилин на Трек А і 30 хвилин на Трек В, але це не жорстке обмеження; розподіляйте час відповідно до власної стратегії.

| Част. | Опис                                            | Трек | Бали | Час    |
|-------|-------------------------------------------------|------|------|--------|
| 1     | Розгортання та розширена розвідка               | A    | 15   | ~15 хв |
| 2     | Перелік сервісів та пошук зачіпки               | A    | 20   | ~20 хв |
| 3     | Експлуатація та отримання користувацького shell | A    | 20   | ~20 хв |
| 4     | Підвищення привілеїв до root                    | A    | 20   | ~15 хв |

| Част. | Опис                                           | Трек | Бали | Час    |
|-------|------------------------------------------------|------|------|--------|
| 5     | Лог-аналіз власної атаки + Sigma-правило       | B    | 10   | ~12 хв |
| 6     | План захисних заходів (3 конфігураційні зміни) | B    | 10   | ~10 хв |
| 7     | Сліпа зона захисту (альтернативний шлях)       | B    | 5    | ~8 хв  |

### Частина 1: Розгортання та розширена розвідка

(15 балів, приблизно 15 хвилин — Трек A)

#### Завдання 1.1 — Розгортання VM (5 балів)

Завантажте Symfonos 1 з VulnHub та імпортуйте у VirtualBox. Режим мережі: Host-Only, спільний з вашою Kali VM. На відміну від Kioptrix-машин, Symfonos не має складнощів із DHCP у Host-Only мережі і зазвичай запускається без втручання. Запишіть IP-адресу цілі через netdiscover чи arp-scan, зробіть скріншот успішного ping.

#### Завдання 1.2 — Повне сканування та початкове враження (10 балів)

Виконайте повне сканування TCP-портів з визначенням версій. Цей крок важливіший, ніж на попередніх машинах, тому що Symfonos має більше сервісів, ніж здається з першого погляду. Якщо ви обмежитеся top-1000 портами (-F або дефолтним Nmap), ви не пропустите головне, але можете пропустити важливі деталі. Рекомендовано сканування -p- з версіями, хоча це займе довше.

Складіть таблицю сервісів. Очікуються щонайменше 4 відкриті порти, серед яких будуть SMB, веб-сервіс і щонайменше один поштовий сервіс. Останнє важливе, зверніть особливу увагу на поштові сервіси, але не припускайте одразу, що ви будете їх атакувати. Запишіть, які сервіси ви бачите, і сформулюйте робочу гіпотезу про те, який із них найбільш перспективний як початковий вектор.

Чесна примітка у стилі ранжування Lab 6: ваша початкова гіпотеза, швидше за все, буде неправильною. Це нормально. Symfonos побудована так, щоб найочевидніший сервіс не був правильним. Зафіксуйте свою гіпотезу письмово ЗАРАЗ, щоб потім ви могли чесно оцінити, наскільки ваше початкове судження збіглося з реальністю.

### Частина 2: Перелік сервісів та пошук зачіпки

(20 балів, приблизно 20 хвилин — Трек A)

Це центральна частина Треку A. Тут ви або знайдете зачіпку, або витратите час даремно. Обидва результати цінні для вашого звіту; друге зобов'язує вас задокументувати невдачу.

#### Завдання 2.1 — Перевірка веб-сервісу (6 балів)

Відкрийте веб-сервер у браузері. Подивіться на головну сторінку. Запустіть Nikto і dirb (або gobuster) з будь-яким стандартним словником. Запишіть знайдені директорії. У цьому конкретному випадку швидкий огляд головної сторінки і кореня веб-сервера може не дати багато це не той вектор, який веде першим. Однак ваш звіт має засвідчити, що ви перевірили цей шлях.

#### Завдання 2.2 — Перелік SMB (10 балів)

SMB на Symfonos — це головний вектор першого рівня, але розповідати про це у лабораторній завданні явно не в наших інтересах. Тому натяк: старі Samba-сервери часто дозволяють анонімну (null) сесію, і саме null-сесія є стандартною першою перевіркою. Використайте  `smbclient -L //<target> -N`, щоб отримати перелік доступних ресурсів. Для кожного знайденого ресурсу спробуйте  `smbclient //<target>/<share> -N` — підключитися анонімно. Зверніть увагу, які саме ресурси відкриті та які з них дозволяють завантажувати файли.

Якщо ви знайшли доступний ресурс із читабельними файлами, уважно прочитайте кожен із них. Не просто завантажте й ідіть далі: відкрийте в редакторі та прочитайте до кінця. Один із файлів на Symfonos містить критичну для всього ланцюжка інформацію, і якщо ви не прочитаєте його уважно, ви просто не знайдете шлях уперед. Це навмисна пастка: walkthrough скаже вам "файл містить X", а вашим завданням є довести у звіті, що ви самі його прочитали і зрозуміли, що X означає у контексті.

Для кожного ресурсу, до якого ви отримали доступ, зафіксуйте у звіті: назву ресурсу, спосіб доступу (анонімно / з обліковими даними), тип знайдених файлів, цитату найважливішого знайденого факту. Фраза "файл містить натяк щодо паролю" недостатня. Потрібна конкретна цитата або чітке пояснення того, що саме ви прочитали.

### **Завдання 2.3 — Поштові сервіси та хибний слід (4 бали)**

Підключіться через netcat до SMTP (25) і POP3 (110) або IMAP (143). Заграбте банери. Опишіть у звіті, що ви побачили, і чому ви вирішили (або не вирішили) продовжувати цей шлях наразі. Якщо ваш звіт показує, що ви витратили 20 хвилин на спроби знайти вразливість у SMTP-сервері як самостійному векторі, це не помилка — це документування невдачі, яку лабораторна від вас вимагає. Головне — рефлексія про те, чому ви туди пішли і що це вас навчило.

Наприкінці частини 2 ви повинні мати чітке розуміння того, який ресурс буде вашим наступним кроком. Якщо ви читали правильні файли з SMB, у вас є специфічна зачіпка щодо одного з компонентів веб-сервера. Якщо у вас немає такої зачіпки після 20 хвилин роботи — ви, ймовірно, пропустили файл у SMB-ресурсі. Поверніться і перечитайте.

### **Частина 3: Експлуатація та користувацький shell**

*(20 балів, приблизно 20 хвилин — Трек А)*

Тут ви застосовуєте знайдену зачіпку. Ця частина складається з двох технічних кроків, і обидва потрібно виконати, щоб отримати shell. Перший крок — знайти вразливість у конкретній частині вебзастосунку. Другий — перетворити читання файлів на виконання коду. Обидва кроки задокументовані у вашому звіті за принципом: що я зробив, чому це спрацювало, що це дало.

### **Завдання 3.1 — Виявлення LFI у веб-застосунку (8 балів)**

На основі натяку з SMB у вас є конкретна підказка, куди дивитися в вебзастосунку. Підключіться до тієї частини застосунку, перегляньте параметри URL. LFI (Local File Inclusion) — це коли вебзастосунок приймає параметр, який використовується як шлях до файлу без належної санітизації. Класичний тест: спробувати прочитати `/etc/passwd` через параметр.

Задокументуйте у звіті: точний URL, який ви використали, точне значення параметра, вміст `/etc/passwd` як доказ читання. Поясніть 2–3 реченнями, чому саме ця форма параметра спрацювала, і яка директива PHP або налаштування веб-сервера могли б цьому перешкодити (ця інформація знадобиться вам у Частині 6).

### **Завдання 3.2 — Log Poisoning для RCE (12 балів)**

LFI дозволяє читати файли, але сама по собі не дає виконувати код. Для переходу від LFI до RCE використовується техніка "log poisoning" (отруєння логів): ви надсилаєте на сервер запит,

у якому шкідливий PHP-код вставляється у поле, що логується сервером, а потім читаєте цей лог через LFI. PHP-інтерпретатор виконує код під час читання логу.

На Symfonos працює щонайменше два класичні варіанти log poisoning. Перший — Apache access.log: ви надсилаєте HTTP-запит з PHP-кодом у заголовку User-Agent, потім через LFI читаєте /var/log/apache2/access.log, і код виконується. Другий — mail log: ви надсилаєте листа через SMTP, у якому PHP-код вставлений у певне поле, потім читаєте його через LFI лог поштового сервера. Обидва варіанти прийнятні; виберіть один.

Для обраного варіанта задокументуйте: точну команду, якою ви вставили код у лог (наприклад, curl з custom User-Agent, або nc з SMTP-діалогом), точний payload (простий webshell на кшталт `<?php system($_GET["c"]); ?>` достатньо), точний URL, через який ви потім виконуєте команди. Зробіть скріншот shell-доступу з виконанням команд whoami, id, hostname. Поясніть 3–4 реченнями, чому саме цей лог-файл був вразливим і що робить цю техніку можливою.

Перетворіть отриманий webshell на повноцінний reverse shell (nc -lnp на Kali + Python or Bash one-liner з цілі). Це не оцінюється окремо, але необхідно для Частини 4 — багато команд ескалації привілеїв не працюють у webshell.

#### **Частина 4: Підвищення привілеїв до root**

*(20 балів, приблизно 15 хвилин — Трек A)*

Ви отримали shell як користувач з обмеженими правами. Тепер потрібно піднятися до root. Symfonos має одну основну стежку ескалації, пов'язану з нестандартним SUID-бінарем. Ваше завдання — знайти його, зрозуміти, що він робить, і використати.

##### **Завдання 4.1 — Перелік SUID-бінарів (6 балів)**

Виконайте команду `find / -perm -u=s -type f 2>/dev/null` з реверс-шелу. Ви побачите стандартний набір SUID-бінарів, а також щонайменше один нестандартний. Запишіть повний список у звіт і позначте, який із бінарів виглядає підозріло (не є частиною стандартного пакета Debian). Якщо ви не впевнені, який саме SUID-бінар є цікавим, порівняйте список із SUID-бінарами вашої власної Kali VM — різниця вас вкаже.

##### **Завдання 4.2 — Аналіз підозрілого бінарля (6 балів)**

Знайшовши нестандартний SUID, не запускайте його одразу наосліп. Спочатку дослідіть. Використайте file, щоб визначити тип. Використайте strings, щоб подивитися на читабельні рядки всередині, часто там видно команди або шляхи, які бінар викликає. Задокументуйте у звіті: шлях, розмір, тип, і щонайменше 3 цікаві рядки з виводу strings. Поясніть, що саме робить цей код, на вашу думку.

##### **Завдання 4.3 — Експлуатація та root-доступ (8 балів)**

Якщо strings показали, що бінар викликає інший виконуваний файл без повного шляху (наприклад, просто curl замість /usr/bin/curl), це вразливість до PATH hijacking. Ви можете створити у директорії, де у вас є право запису, шкідливий виконуваний файл з тією самою назвою, додати цю директорію на початок PATH, і запустити SUID-бінар. SUID-бінар запустить ваш шкідливий файл з правами root.

Задокументуйте точні кроки: де ви створили фальшивий бінар, що в нього помістили (зазвичай /bin/sh з `setuid(0)`), як модифікували PATH, як запустили SUID-бінар. Зробіть скріншот root shell з виконанням whoami, id, cat /etc/shadow (для доказу, що у вас реальний root, а не просто EUID-підміна). Напишіть 2–3 речення про те, чому ця атака спрацювала — чому небезпечно викликати бінари без повного шляху в SUID-контексті.

### ПЕРЕХІД ДО ТРЕКУ В

Якщо ви завершили Частини 1–4 і отримали root, ви вже маєте повний бал за Треком А (75/100). Решта документа, це частини 5–7, це Трек В, розширення на 25 балів для оцінки А/А-.

Трек В вимагає, щоб ви подивилися на власну атаку з іншого боку. Клієнт запитує: що б я побачив у своїх логах? Що мені змінити, щоб це не повторилося? Де ще я вразливий, навіть якщо зроблю ці зміни? Трек В не можна виконати, читаючи walkthroughs — він вимагає ВАШИХ даних з ВАШОЇ атаки на ЦЬОМУ сервері.

Для треку В вам потрібен доступ root до цілі (для читання логів у /var/log). Якщо ви втратили reverse shell, відновіть його перед початком частини 5.

### Частина 5: Лог-аналіз власної атаки та Sigma-правило

(10 балів, приблизно 12 хвилин — Трек В)

Захисник, який прокинеться наступного ранку після вашої атаки, матиме лише логи. Ваше завдання полягає в тому, щоб подивитися на ці самі логи зараз і побачити те, що побачить він. Це вимагає пройти власну атаку задом наперед: від відомого моменту отримання shell назад до першого запиту.

#### Завдання 5.1 — Збір артефактів власної атаки (5 балів)

З root shell на цілі зберіть лог-рядки, які згенерувала ваша атака. Перше місце для перевірки — /var/log/apache2/access.log (або той лог, який ви використовували для log poisoning). Знайдіть рядки з IP-адресою атакуючого. Друге місце — /var/log/samba/ (якщо ви взаємодіяли з SMB — ці логи можуть бути або не бути). Третє — /var/log/auth.log (якщо робили спроби SSH).

Побудуйте часову таблицю у звіті: стовпці — час (timestamp з логу), джерело (який лог-файл), фрагмент лог-рядка (скорочений до суттєвого), що саме зафіксовано (у ваших словах). Мінімум 5 рядків. Зверніть особливу увагу на рядки log poisoning: ваш шкідливий User-Agent із PHP-кодом має бути помітний у логах.

#### Завдання 5.2 — Sigma-правило для своєї атаки (5 балів)

Напишіть одне Sigma-правило, яке виявляло б СПЕЦИФІЧНО ту техніку log poisoning, яку ви застосували. Не загальне правило "detect LFI" — такі правила існують у кожній бібліотеці Sigma і не дадуть вам балів. Ваше правило має цільово виявляти патерн: запит до певної сторінки з PHP-подібним вмістом у User-Agent, супроводжуваний подальшим запитом до тієї ж сторінки, що читає access.log. Використайте реальні дані з вашої часової таблиці (Завдання 5.1).

Шаблон Sigma-правила:

title: [Коротка назва вашої атаки]

description: [1-2 речення про те, що саме це правило виявляє]

author: [ваше ім'я]

logsource:

product: [apache / samba / linux]

category: [webserver / file\_share / ...]

detection:

selection:

[поле]: [шаблон, який ви бачили у власних логах]

condition: selection

level: [low / medium / high / critical]

Після правила напишіть абзац (3–4 речення): які false positives може генерувати ваше правило (тобто які легітимні запити можуть виглядати схоже), і як ви б це обмежили у продакшн-середовищі. Це не теоретичне питання — кожне добре правило має відомий рівень false positives, і зрілий аналітик це документує.

#### Частина 6: План захисних заходів

(10 балів, приблизно 10 хвилин — Трек В)

##### Завдання 6.1 — Три конфігураційні зміни (7 балів)

Запропонуйте 3 конкретні конфігураційні зміни, які б зламали ваш ланцюжок атаки. Кожна зміна має бути: адресною (вказані файл і директива), конкретною (вказане поточне значення і нове значення), і виправданою (пояснено, який саме крок вашої атаки вона зламала б).

Використайте таблицю нижче. Пріоритет від 1 до 3, де 1 — найбільш ефективна зміна (та, що ламає найбільше кроків атаки за найменшу вартість для бізнесу). Стовець "Що ламається у вашій атаці" повинен посилатися на конкретний крок: "зламає Частину 2.2", "зламає Частину 3.2", "зламає Частину 4.3".

| Пріор. | Файл / Сервіс | Директива | Поточне знач. → Нове | Що ламається у вашій атаці |
|--------|---------------|-----------|----------------------|----------------------------|
| 1      | ...           | ...       | ... → ...            | ...                        |
| 2      | ...           | ...       | ... → ...            | ...                        |
| 3      | ...           | ...       | ... → ...            | ...                        |

##### Завдання 6.2 — Одна критична зміна (3 бали)

З трьох запропонованих змін виберіть одну — ту, що, якби її застосували, зламала б ваш ланцюжок на найранішому етапі, і яку найлегше імплементувати у середовищі клієнта. Напишіть абзац (3–4 речення) з двома частинами. Перша: чому саме ця зміна критична (що сталося б, якби всі три не застосували, а цю одну — так). Друга: яку ціну платить бізнес за цю зміну (незручність

для користувачів, необхідність перенавчання, вартість, потенційні побічні ефекти). Без розуміння ціни рекомендація звучить як вимога; з розумінням ціни — як професійна порада.

Частина 7: Сліпа зона захисту

(5 балів, приблизно 8 хвилин — Трек В)

Це коротка, але вимоглива частина. Її мета — перевірити, чи ви розумієте обмеженість власних рекомендацій.

Уявіть, що клієнт погодився на дві з трьох ваших змін, але третю відклав з бізнес-міркувань (наприклад, стара плагін-залежність у веб-застосунку, яку дорого замінити; або SMB з анонімним доступом потрібен для інтеграції з іншою системою). Напишіть абзац (150–200 слів) про альтернативний шлях атаки, який залишиться відкритим у такому сценарії. Це не вимагає реального виконання нової атаки це гіпотеза, підкріплена тим, що ви бачили під час Треку А.

Структура абзацу: перше речення, яке з трьох рекомендацій відклали. Друге речення — яка частина поверхні атаки залишилася недоторканою. Далі 3–4 речення: який гіпотетичний вектор ви бачите, спираючись на сервіси з вашої частини 1, які не торкалися жодною з рекомендацій. Останнє речення це новий вектор, якого не було в оригінальній атаці, чи варіація тієї, яку ви провели?

Хороша відповідь покаже, що ви розумієте принцип defense-in-depth: одна зміна ніколи не закриває всю поверхню, і добрий пентест-звіт допомагає клієнту бачити залишкові ризики, а не лише усунути. Слабка відповідь буде повторювати те, що ви вже написали у частині 6.

#### **Важливі примітки**

Symfonos 1 — Debian-based Linux середини 2010-х. Мережевих проблем у VirtualBox у неї зазвичай немає. Якщо цільова машина не завантажується у графічному режимі, це очікувано: Symfonos часто завантажується просто у термінальний login-prompt без GUI, тому netdiscover — ваш основний спосіб її знайти.

На відміну від Kioptrix-машин, Symfonos навмисно вставляє хибні сліди: SSH, SMTP, IMAP. Якщо ви витратили 30 хвилин на Hydra замість SSH і нічого не отримали, це очікуваний результат, який потрібно зафіксувати як невдачу. Walkthroughs в інтернеті мовчать про те, скільки часу їхні автори втратили на тупики, тому що пишуть постфактум.

Треки. Трек А — обов'язковий, 75 балів, максимум В+ / А-. Трек В — додатковий, ще 25 балів, потрібен для А. Трек В оцінюється лише за умови, що Трек А виконано щонайменше на 50/75 балів.

Вся робота виконується в ізолюваному середовищі Host-Only. Не скануйте машини поза межами VirtualBox. Див. Розділ 6 силабусу щодо політики використання AI та академічної доброчесності.

Що здавати

Єдиний звіт у форматі DOCX з усіма завданнями, скріншотами, таблицями та аналізом. Назвіть файл Lab7\_Прізвище\_Група.docx. Якщо ви виконали лише Трек А, зазначте це на титульній сторінці звіту ("Виконано: Трек А"). Якщо виконали обидва — «Виконано: Трек А + Трек В». Здайте звіт до дедлайну.

Орієнтовний обсяг власного тексту: 1500–2000 слів для Треку А, додаткові 800–1200 слів для Треку В. Звіт, який складається здебільшого зі скріншотів без тексту, буде оцінено за нижньою межею для кожного розділу

Успіху.